Whenever you run your application, either using the `dotnet run` command or Ctrl-F5 in Visual Studio, the first thing that happens is a build, which results in the source code being compiled into executables and placed in the Bin folder. The compiled output is not, however, something you can simply deploy to another environment, such as a web server. Another step is required to prepare your application for deployment. You need to *publish* it. This is achieved using the `dotnet publish` command or the publishing tooling within Visual Studio. In this section, we will explore the various configuration options available and then explore the published output. We will also consider some troubleshooting tips that will help resolve some of the most common issues that arise when the published application is finally deployed to a produc-tion server.

14.4.1 Self-contained and framework-dependent

The first configuration decision you need to make is choosing between publishing a *self-contained* application or a *framework-dependent* one. Both options result in your application code being built into .dll files, but the self-contained version includes the .NET Core runtime files as well. You will use this option if you are not sure whether the necessary .NET Core runtime is installed on the target machine. This option results in a relatively large published output. Your CityBreaks application, when published as a self-contained application, weighs in at just over 400 separate files (not including the wwwroot folder contents), occupying around 125 MB of space.

It is possible to use trimming to reduce the overall published output of a self-contained deployment. Trimming scans your code and removes any runtime assemblies that are not called into. You activate trimming by adding the following entry to your project file, just below the `TargetFramework` node: